



Senate Building Decoration (decoration)

Estás envolvido na construção do novo e imponente edifício do Senado, e uma tarefa chave é escolher a sua decoração central. O Senado quer a a decoração mais cara possível. Contudo, os Plebeus também querem fazer parte da escolha, e estão preocupados que uma decoração muito cara venha a resultado em impostos mais altos. Desse modo, querem selecionar a decoração mais barata.

Para decidir, um array de N potenciais decorações é apresentado, com as decorações distribuídas numa linha. Cada decoração, indexada de 0 a $N - 1$, tem um custo específico, indicado por A_i .

Após longas discussões, foi escolhido o seguinte método de decisão. O Senado e os Plebeus jogam por turnos, alternando entre quem joga em cada turno. **O Senado joga primeiro.** Os seus objetivos são opostos:

- O Senado pretende garantir que a decoração final tenha o custo **mais alto** possível.
- Os Plebeus pretendem que a decoração final tenha o custo **mais baixo** possível.

Em cada turno, o jogador atual (Senado ou Plebeus) considera as decorações que ainda estão disponíveis. Uma jogada consiste em selecionar um bloco contíguo (um subarray) dessas decorações; o bloco selecionado **não pode ser o bloco atual inteiro** e deve ter **pelo menos metade** (arredondando para cima) do comprimento do bloco atual de decorações. Todas as decorações que estiverem **fora** desse bloco escolhido são então removidas.

Este processo continua, com a fila de decorações a diminuir após cada turno, até que reste apenas uma única decoração. O custo desta decoração final é o resultado. A tua tarefa é prever este custo, assumindo que tanto o Senado como os Plebeus jogam de forma ótima para atingir os seus respetivos objetivos.

Implementação

Tens de submeter um único ficheiro de código `.cpp`.



Entre os ficheiros anexos deste problema encontra-se um template `decoration.cpp` com um exemplo de implementação.

Tens de implementar a seguinte função:

C++

```
int decorate(int N, vector<int> A)
```

- O inteiro N representa o número inicial de potenciais decorações.
- O array A , indexado de 0 a $N - 1$, contém os valores $A_0, A_1, \dots, A_{N-1}$, onde A_i é o custo da i -ésima decoração.
- A função deve devolver o custo da decoração selecionada.

Avaliador de Exemplo

Entre os ficheiros anexos deste problema, encontrarás uma versão simplificada do avaliador usado numa submissão real, que poderás usar para testar as soluções localmente. O avaliador lê dados de `stdin`, chama a função `decorate` e escreve de volta em `stdout` usando o seguinte formato.

O input deve ser formado por 2 linhas, contendo:

- Linha 1: o inteiro N .
- Linha 2: N inteiros separados por espaços, representando os elementos $A_0, A_1, \dots, A_{N-1}$.

O output é constituído por uma única linha, contendo os valores devolvidos pela função **decorate**.

Restrições

- $1 \leq N \leq 5000$.
- $0 \leq A_i \leq 1\,000\,000\,000$.

Pontuação

O teu programa será testado com vários casos de teste agrupados em subtarefas. Para obter a pontuação de uma subtarefa, o teu programa tem de resolver corretamente todos os seus casos de teste.

- **Subtarefa 0 [0 pontos]**: Casos de exemplo.
- **Subtarefa 1 [12 pontos]**: $N \leq 1500$, A é não decrescente (i.e. para $i < j$, $A_i \leq A_j$).
- **Subtarefa 2 [4 pontos]**: $N \leq 10$, $A_i \leq 1000$
- **Subtarefa 3 [22 pontos]**: $N \leq 250$
- **Subtarefa 4 [26 pontos]**: $N \leq 800$
- **Subtarefa 5 [33 pontos]**: $N \leq 1500$
- **Subtarefa 6 [3 pontos]**: Nenhuma restrição adicional.

Exemplos

stdin	stdout
5 0 1 2 3 4	3
5 4 3 1 1 5	3

Explicação

No primeiro exemplo, pode ser mostrado que a seguinte sequência de movimentos é ótima:

- Primeira jogada do Senado: o único subarray que maximiza o pior resultado possível para eles: $[2, 3, 4]$.
- Resposta dos Plebeus: De $[2, 3, 4]$ eles têm de escolher um subarray de tamanho pelo menos dois; escolher $[2, 3]$ minimiza o eventual resultado.
- Finalmente o Senado escolhe 3.

No segundo exemplo:

- Primeira jogada do Senado: a única abertura que mantém o valor garantido acima de 2 é o bloco $[4, 3, 1]$ (tamanho 3, pelo menos metade de 5 e sem ser a linha inteira).
- Resposta dos Plebeus: De $[4, 3, 1]$ eles têm de escolher um subarray de tamanho pelo menos dois; escolher $[3, 1]$ minimiza o eventual resultado.
- Finalmente o Senado escolhe 3.

Qualquer outra primeira jogada (ex: $[3, 1, 1]$, $[1, 1, 5]$ ou um bloco de tamanho 4), deixam os Plebeus forçar um resultado de 1, e portanto jogadas ótimas de ambos os lados terminam com um custo de 3.